

Consola Retro SDK — Sistema, Instalación Automatizada e Integración

Carrillo Viña Sebastián - 318060871

Fundamentos de Sistemas Embebidos

Tutorial individual — Módulos: `usb_monitor.py` · `main.py` · `instalar.sh`

1. Objetivo

Implementar la integración principal de una consola retro embebida en Raspberry Pi, asegurando su arranque automático, detección de memorias USB y ejecución funcional de los módulos del sistema.

Desarrollar e integrar los módulos *main.py*, *usb_monitor.py* e *instalar.sh* para coordinar el inicio del sistema, automatizar la instalación de dependencias, configurar el servicio de arranque con `systemd` y permitir la detección y copia de ROMs desde una memoria USB de forma segura.

2. Lista de materiales

Se enumera el hardware necesario para reproducir el sistema.

Tabla 1. Hardware requerido

Componente	Cantidad	Notas
Raspberry Pi 4 Model B (2 GB o más)	1	ARM Cortex-A72 64-bit
Tarjeta microSD (16 GB+, clase 10)	1	Sistema operativo y ROMs
Monitor o TV con entrada HDMI	1	Resolución mínima 720p
Cable micro-HDMI a HDMI	1	Incluido en kits oficiales
Fuente de alimentación 5 V / 3 A USB-C	1	Se recomienda fuente oficial
Gamepad USB compatible con SDL2	1	Compatible con SDL2 (ej. Xbox, genérico)
Memoria USB (FAT32 o exFAT)	1	Para carga de ROMs adicionales

En cuanto al software se requiere:

- Raspberry Pi OS Lite (64-bit), sin entorno de escritorio
- Python 3.11 o superior
- pygame 2.x (python3-pygame)
- pyudev — bindings Python para libudev (python3-pyudev)
- mednafen — emulador multi-sistema (`apt install mednafen`)
- SDL2 con driver kmsdrm (libsdl2-2.0-0, libsdl2-dev)

- git, curl y unzip — para la instalación automatizada
- ROMs libres de derechos para NES, SNES y Game Boy Advance

3. Antecedentes teóricos

3.1 Integración de sistemas embebidos

Un sistema embebido es un sistema de cómputo diseñado para cumplir una función específica dentro de un dispositivo mayor. A diferencia de una computadora de propósito general, normalmente trabaja con recursos limitados y debe operar de manera autónoma, estable y confiable. [1].

Un principio fundamental de diseño en sistemas embebidos es que el software debe arrancar de forma autónoma sin intervención humana al recibir alimentación. Esto requiere configurar el sistema operativo para que lance la aplicación automáticamente como parte del proceso de arranque, antes de que cualquier sesión de usuario esté disponible [1].

3.2 systemd y servicios de arranque

Para lograr que la consola se ejecute automáticamente al encender la Raspberry Pi, se utiliza systemd, el sistema de inicialización y administración de servicios usado en muchas distribuciones Linux modernas. Mediante un archivo de servicio, es posible indicar qué programa debe ejecutarse, con qué usuario, desde qué directorio y qué hacer si el proceso falla. Esto es importante en el proyecto porque permite que la consola arranque sin intervención manual y que pueda reiniciarse automáticamente si ocurre un error [2].

3.3 udev y detección de hardware

udev es el gestor de dispositivos del kernel Linux. Cuando se conecta una memoria USB, el kernel genera un evento uevent que udev procesa. pyudev permite escuchar estos eventos desde Python mediante un monitor asíncrono bloqueante: el hilo de monitoreo queda en `monitor.poll()` consumiendo 0% de CPU hasta que llega un evento, sin ciclos desperdiciados [8]. Este enfoque es mucho más eficiente que el polling (revisar periódicamente si aparecen dispositivos nuevos).

3.4 Montaje de sistemas de archivos

Para acceder al contenido de una memoria USB en Linux es necesario montar su sistema de archivos en un directorio del árbol. El parámetro `-o ro` monta el dispositivo en modo solo lectura, protegiendo los datos del USB contra escrituras accidentales. Antes de retirar el USB es obligatorio desmontarlo con `umount` para garantizar que el kernel haya escrito en disco todos los datos pendientes en su caché [5].

3.5 Programación con hilos y exclusión mutua

El monitoreo de USB se ejecuta en un hilo separado (`threading.Thread`) configurado como `daemon=True` para que termine automáticamente cuando el proceso principal termina. La comunicación entre el hilo de monitoreo y el loop principal se realiza a través de la variable booleana `_usb_nuevo` protegida por un `threading.Lock`. Sin el lock, si ambos hilos accedieran a esta variable simultáneamente se produciría una condición de carrera [6].

3.6 Scripts bash de instalación automatizada

Un script bash de instalación automatizada reúne todos los pasos de configuración en un único archivo ejecutable, eliminando errores humanos y permitiendo reproducir el entorno de forma idéntica. La directiva set -euo pipefail es fundamental: -e termina el script ante cualquier error, -u falla si se usa una variable no definida, y -o pipefail detecta errores dentro de comandos encadenados [5].

3.7 Visión general del sistema

La Tabla 2 muestra todos los módulos del sistema y cómo cada uno contribuye al conjunto. Los módulos de interfaz (pantalla.py, arranque.py, galeria.py) manejan el renderizado; los módulos de entrada y emulación (entrada.py, emulador.py) gestionan el gamepad y mednafen; y los módulos de integración (usb_monitor.py, main.py, instalar.sh) conectan todo y garantizan el arranque autónomo.

Tabla 2. Módulos del sistema Consola Retro SDK

Módulo	Archivo	Función en el sistema
Pantalla	pantalla.py	Renderizado de toda la interfaz gráfica con pygame
Arranque	arranque.py	Secuencia de animación al encender (fade-in/out)
Galeria	galeria.py	Loop principal; catálogo de ROMs y menú de navegación
Entrada	entrada.py	Captura de gamepad/teclado y mapeo a acciones lógicas
Emulador	emulador.py	Gestión del framebuffer y lanzamiento de mednafen
MonitorUSB	usb_monitor.py	Detección de USB en hilo daemon y copia de ROMs
Principal	main.py	Punto de entrada; instancia y conecta todos los módulos

3.8 Interfaz gráfica con pygame y doble buffer

pygame es la biblioteca Python que gestiona el renderizado del sistema sobre SDL2. Sigue el paradigma de doble buffer: todas las operaciones de dibujo se hacen en memoria y solo con pygame.display.flip() se transfieren a la pantalla, eliminando el parpadeo. En Raspberry Pi OS Lite, SDL2 usa el driver kmsdrm para escribir directamente al framebuffer sin servidor X, configurado mediante la variable SDL_VIDEODRIVER=kmsdrm. La clase Pantalla encapsula toda esta lógica; ningún otro módulo llama a pygame directamente. La animación de arranque usa interpolación de alpha (Surface.set_alpha()) de 0 a 255 en 1.5 segundos a 60 fps. El menú de galería usa una ventana deslizante de 14 elementos centrada en el ítem seleccionado: inicio = max(0, indice - V // 2), fin = min(total, inicio + V) [3].

3.9 Captura de entrada y gestión del framebuffer

La clase Entrada mapea los controles físicos del gamepad a acciones lógicas mediante el estándar HID. Los ejes analógicos usan una zona muerta de 0.5 para evitar el drift, y el sistema detecta conexión y desconexión del gamepad en caliente. La clase Emulador gestiona la transición al emulador mednafen: dado que KMS/DRM garantiza acceso exclusivo al framebuffer, antes de lanzar mednafen se llama a `pygame.display.quit()` para liberarlo, se espera 1 segundo, se inicia mednafen con `subprocess.Popen()` y se guarda la referencia al proceso en `self.proceso`. Esto es precisamente lo que permite a MonitorUSB terminar mednafen con `proceso.terminate()` cuando se detecta un USB con ROMs durante el juego. Al regresar, `pygame` se reinicializa y recupera el framebuffer [2][4].

4. Advertencias y cuidado de componentes

4.1 Seguridad durante la instalación

- El script `instalar.sh` debe ejecutarse con `sudo`. Revisar su contenido antes de ejecutarlo ya que modifica archivos del sistema como `/etc/sudoers.d/`, `/etc/systemd/system/` y `/boot/firmware/cmdline.txt`.
- Realice un respaldo de la tarjeta microSD antes de ejecutar el script. Una configuración incorrecta puede impedir el arranque del sistema.
- Habilite SSH antes de ejecutar el script en modo remoto para poder recuperar el control del sistema si algo falla.

4.2 Seguridad de la tarjeta microSD

- El sistema está configurado para apagarse siempre mediante `sudo shutdown -h now` antes de cortar la alimentación. Este paso es crítico para la integridad del sistema de archivos.
- El archivo `/boot/NO_CONSOLA` deshabilita el arranque automático, útil para mantenimiento sin modificar la configuración del servicio.

4.3 Cuidado con memorias USB

- Formatear las memorias USB en FAT32 o exFAT para garantizar compatibilidad. El sistema monta en modo solo lectura, por lo que no modifica el contenido del USB.
- Retirar el USB únicamente después de que la consola muestre el mensaje de ROMs copiadas y actualice la galería.

5. Descripción de los módulos de sistema e integración

5.1 Estructura del proyecto

La Tabla 3 describe la estructura de directorios del proyecto.

Tabla 3. Estructura de directorios del proyecto

Ruta	Contenido
<code>src/main.py</code>	Punto de entrada; instancia y conecta todos los módulos

src/config/configuracion.json	Resolución, rutas de assets y mapeo de emuladores
src/config/mednafen/mednafen.cfg	Configuración pre-ajustada de mednafen para Raspberry Pi
src/roms/nes/	ROMs de NES (.nes)
src/roms/snes/	ROMs de Super Nintendo (.smc, .sfc)
src/roms/gba/	ROMs de Game Boy Advance (.gba)
src/assets/imagenes/	Logo de arranque e imágenes de controles por consola
src/assets/sonidos/	Archivo WAV de sonido de arranque
src/instalar.sh	Script de instalación automatizada completa

5.2 Módulo main.py — punto de entrada

main.py es el único archivo que se ejecuta directamente. Su responsabilidad es instanciar todos los módulos en el orden correcto y conectar sus dependencias. No contiene lógica de negocio: delega toda la funcionalidad a los módulos especializados.

1. `_instalar_config_mednafen()`: copia mednafen.cfg al directorio ~/.mednafen garantizando la configuración correcta en cada arranque.
2. `cargar_configuracion()`: lee el JSON y termina con `sys.exit(1)` si el archivo no existe o tiene errores de sintaxis.
3. Instanciar Pantalla con los parámetros de resolución del JSON (inicializa pygame).
4. Instanciar Entrada y MonitorUSB de forma independiente.
5. Instanciar Emulador con referencia a Pantalla, ruta de imágenes y mapeo de emuladores.
6. Registrar la referencia del Emulador en MonitorUSB con `set_emulador()`.
7. Instanciar Galeria con referencias a todos los módulos anteriores.
8. Instanciar y ejecutar Arranque (secuencia de logo con fade).
9. Iniciar galeria.ejecutar() que no retorna hasta el apagado.
10. En el bloque finally, llamar a pantalla.cerrar() para liberar recursos de pygame.

5.3 Módulo usb_monitor.py — Clase MonitorUSB

La clase MonitorUSB gestiona la detección de memorias USB, su montaje, la copia de ROMs y la comunicación con el loop principal y el emulador, todo en un hilo separado para no interferir con la interfaz gráfica. La Tabla 4 describe todos sus métodos.

Tabla 4. Métodos de la clase MonitorUSB

Método	Descripción
<code>__init__()</code>	Crea directorio de montaje, inicia hilo de monitoreo daemon
<code>set_emulador(emulador, ruta)</code>	Registra referencia al emulador para terminarlo si hay USB durante el juego
<code>hay_nuevo_usb()</code>	Devuelve True y resetea bandera si hay USB listo para procesar

<code>copiar_roms_de_usb(ruta_local)</code>	Copia ROMs del USB al proyecto evitando duplicados; desmonta al terminar
<code>_monitorear_udev()</code>	Hilo principal: escucha eventos de bloque con pyudev
<code>_monitorear_polling()</code>	Fallback sin pyudev: revisa <code>/dev/sd*</code> cada 2 segundos
<code>_montar(dispositivo)</code>	Monta el dispositivo USB en punto de montaje fijo en modo solo lectura
<code>_desmontar()</code>	Desmonta el USB de forma segura después de copiar
<code>_buscar_archivos(ruta, exts)</code>	Búsqueda recursiva de archivos por extensión con glob

5.3.1 Arquitectura de doble hilo

MonitorUSB trabaja con dos hilos principales. El primero es el hilo principal de la aplicación, donde se ejecuta la interfaz de la galería. El segundo es un hilo de monitoreo que permanece atento a la conexión de memorias USB.

Esta arquitectura evita que la consola se congele mientras espera la inserción de un dispositivo. El hilo de monitoreo detecta el USB y actualiza una bandera interna llamada `_usb_nuevo`, mientras que la galería revisa esa bandera desde su ciclo principal mediante el método `hay_nuevo_usb()`.

5.3.2 Fallback de polling

Este método revisa periódicamente la existencia de dispositivos como `/dev/sda1` o `/dev/sdb1`. Aunque consume más recursos que el enfoque basado en eventos, permite que el sistema siga funcionando en entornos donde pyudev no pueda utilizarse correctamente.

5.4 Script instalar.sh

El script `instalar.sh` configura el sistema completo desde una instalación limpia de Raspberry Pi OS Lite. Se ejecuta con un único comando `curl` desde internet. Detecta automáticamente el usuario real aunque se ejecute con `sudo`, instala todas las dependencias, descarga el código fuente y los assets, configura el servicio `systemd` y reinicia la Raspberry Pi al finalizar.

5.5 Módulos de interfaz gráfica

`pantalla.py` define la clase `Pantalla`, que inicializa `pygame` en modo pantalla completa con el driver `kmsdrm`, define tres tamaños de fuente monoespaciada y expone métodos de renderizado (`limpiar`, `dibujar_texto`, `dibujar_imagen`, `reproducir_sonido`, `actualizar`). Ningún otro módulo llama a `pygame` directamente. `arranque.py` ejecuta la secuencia de presentación al encender: carga el logo, reproduce el sonido de moneda y ejecuta un fade-in/out de 1.5 segundos mediante interpolación de alpha. `galeria.py` contiene el loop principal a 30 fps: escanea las carpetas de ROMs, dibuja el menú con ventana deslizante de 14 ítems, lanza el emulador al seleccionar un juego y gestiona el apagado seguro con `sudo shutdown -h now` previa confirmación del usuario [3].

5.6 Módulos de entrada y emulación

entrada.py mapea los controles físicos del gamepad a acciones lógicas mediante constantes de cadena. Incluye zona muerta de 0.5 para los ejes analógicos y soporte de hot-plug. emulador.py orquesta el ciclo completo de una sesión: muestra la pantalla de controles con imagen por consola y texto parpadeante, libera el framebuffer con `pygame.display.quit()`, lanza mednafen con `subprocess.Popen()` guardando la referencia al proceso en `self.proceso` (lo que permite a MonitorUSB terminarlo durante el juego), y al regresar reinicializa pygame. Las tres consolas soportadas son NES (-force_module nes), SNES (-force_module snes) y GBA (-force_module gba) [2][9].

6. Instalación del sistema

El sistema se instala ejecutando un único comando desde la terminal SSH de la Raspberry Pi. El script `instalar.sh` automatiza todos los pasos necesarios para tener la consola funcionando desde cero en una instalación limpia de Raspberry Pi OS Lite:

```
sudo bash -c "$(curl -fsSL https://raw.githubusercontent.com/nasaruke/SDK_RetroGames/main/src/instalar.sh)"
```

La Tabla 2 resume los pasos que ejecuta el script en orden:

Tabla 2. Pasos del script de instalación `instalar.sh`

Paso	Acción	Descripción
0	Actualizar sistema	apt update e instalación de git, python3, curl
1	Dependencias SDL	libsdl2, libsdl2-mixer, libsdl2-image, libsdl2-ttf
2	pygame y pyudev	python3-pygame, python3-pyudev vía apt
3	mednafen	Instalación y enlace simbólico en /usr/local/bin
4	Permisos de usuario	Agregar usuario a grupos video, audio, input, render
5	Estructura de directorios	Crear carpetas roms/nes, roms/snes, roms/gba
6	Código fuente	Clonar repositorio GitHub y copiar archivos .py
7	Assets y ROMs	Descargar y descomprimir desde branch instalaciones
8	Sudoers	Permitir mount, umount y shutdown sin contraseña
9	Configuración mednafen	Copiar mednafen.cfg preconfigurado a ~/.mednafen
10	Variables SDL en .bashrc	SDL_VIDEODRIVER=kmsdrm, SDL_AUDIODRIVER=alsa
11	Servicio systemd	Crear y habilitar consola-retro.service con autoarranque
12	TTY de mantenimiento	Habilitar getty@tty2 para acceso con Ctrl+Alt+F2
13	Silenciar arranque	Agregar quiet loglevel=0 logo.nologo a cmdline.txt

El script detecta automáticamente el usuario real aunque se ejecute con sudo, instala todas las dependencias, descarga el código fuente y los assets desde GitHub, configura el servicio

systemd para el arranque automático y reinicia la Raspberry Pi al finalizar. El proceso completo tarda aproximadamente 5 a 10 minutos dependiendo de la velocidad de la conexión a internet.

Una vez instalado, el sistema arranca automáticamente cada vez que se enciende la Raspberry Pi. Para acceder al modo de mantenimiento sin que la consola inicie:

```
sudo touch /boot/NO_CONSOLA && sudo reboot
```

Para regresar al modo consola:

```
sudo rm /boot/NO_CONSOLA && sudo reboot
```

6.1 TTY de mantenimiento.

Aunque el objetivo es que la Raspberry Pi funcione como una consola dedicada, también es importante conservar una forma de acceder al sistema para mantenimiento. Para esto se habilita una terminal secundaria mediante `getty@tty2`.

Esta terminal puede abrirse con la combinación: `Ctrl + Alt + F2`.

Desde ahí es posible iniciar sesión, revisar archivos, detener el servicio, consultar logs o realizar cambios de configuración sin depender únicamente de SSH.

7. Desarrollo de los módulos

El archivo `main.py` funciona como el punto de entrada del sistema. Antes de iniciar la interfaz de la consola, este módulo se encarga de preparar la configuración general y crear las instancias necesarias para que los demás módulos trabajen en conjunto.

7.1 `main.py` — carga de configuración e instanciación

```
RUTA_BASE = os.path.dirname(os.path.abspath(__file__))

def cargar_configuracion(ruta_config: str) -> dict:
    if not os.path.exists(ruta_config):
        print(f'[main] ERROR: {ruta_config} no encontrado')
        sys.exit(1)
    with open(ruta_config, 'r', encoding='utf-8') as archivo:
        try:
            return json.load(archivo)
        except json.JSONDecodeError as e:
            print(f'[main] ERROR en configuracion.json: {e}')
            sys.exit(1)
```

La función `construir_ruta()` genera rutas absolutas a partir de la ruta base del script, garantizando que las rutas funcionen independientemente del directorio desde el que se lanza el proceso (crítico cuando systemd lanza el proceso desde `/`):

```
def construir_ruta(config: dict, *claves) -> str:
    valor = config
    for clave in claves:
        valor = valor[clave]
```



```
return os.path.join(RUTA_BASE, valor)
```

7.2 MonitorUSB — hilo de monitoreo con pyudev

```
self._hilo = threading.Thread(
    target=self._monitorear_udev,
    daemon=True # termina automáticamente con el proceso principal
)
self._hilo.start()
```

El método `_monitorear_udev()` filtra por subsistema 'block' y tipo 'partition' para recibir solo eventos de memorias USB, ignorando teclados y gamepads:

```
def _monitorear_udev(self):
    import pyudev
    context = pyudev.Context()
    monitor = pyudev.Monitor.from_netlink(context)
    monitor.filter_by(subsystem='block', device_type='partition')
    for device in iter(monitor.poll, None): # bloquea hasta evento
        if device.action == 'add':
            time.sleep(1) # esperar que el kernel configure el
dispositivo
            if self._montar(device.device_node):
                if self._emulador_ref and self._emulador_ref.proceso:
                    self._emulador_ref.proceso.terminate()
                    self._emulador_ref.proceso.wait()
                with self._lock:
                    self._usb_nuevo = True
```

7.3 MonitorUSB — copia de ROMs sin duplicados

```
def copiar_roms_de_usb(self, ruta_roms_local: str) -> int:
    total = 0
    for consola, extensiones in self.EXTENSIONES_POR_CONSOLA.items():
        roms_en_usb = self._buscar_archivos(self.PUNTO_MONTAJE,
extensiones)
        destino = os.path.join(ruta_roms_local, consola)
        os.makedirs(destino, exist_ok=True)
        ya_existen = {f.lower() for f in os.listdir(destino)}
        for ruta_rom in roms_en_usb:
            nombre = os.path.basename(ruta_rom)
            if nombre.lower() in ya_existen:
                continue # omitir duplicado
            shutil.copy2(ruta_rom, os.path.join(destino, nombre))
            ya_existen.add(nombre.lower())
            total += 1
    self._desmontar()
    return total
```

7.4 instalar.sh — servicio systemd

El script genera el archivo de servicio dinámicamente con las variables del usuario detectado:

```
[Unit]
Description=Consola Retro SDK
After=multi-user.target sound.target local-fs.target
```

```

ConditionPathExists=!/boot/NO_CONSOLA

[Service]
Type=simple
User=$USUARIO
WorkingDirectory=$RUTA_SRC
Environment=SDL_VIDEODRIVER=kmsdrm
Environment=SDL_AUDIODRIVER=alsa
ExecStart=/usr/bin/python3 $PYTHON_SCRIPT
StandardOutput=append:$LOG_FILE
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target

```

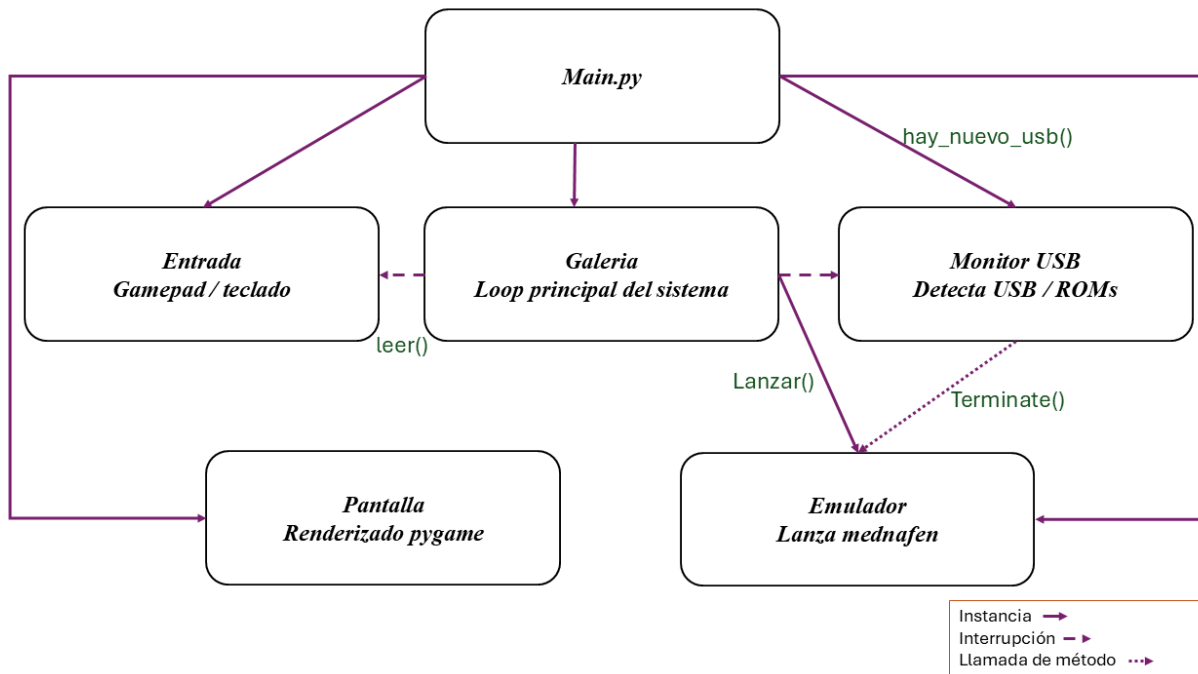
8. Integración del sistema completo

La Figura 1 ilustra cómo main.py conecta todos los módulos del sistema. El loop de Galeria es el único componente que corre indefinidamente; todos los demás módulos son llamados desde él o desde sus dependencias.

El flujo de inserción de USB durante el juego es el siguiente:

11. El hilo de MonitorUSB detecta el evento 'add' de pyudev.
12. Llama a `_montar()` para montar el USB en modo solo lectura.
13. Si `Emulador.proceso` no es `None`, llama a `proceso.terminate()` y `.wait()`.
14. Establece `_usb_nuevo = True` dentro del lock.
15. En la siguiente iteración del loop de Galeria, `hay_nuevo_usb()` devuelve `True`.
16. La galería muestra el mensaje, copia las ROMs y actualiza el catálogo.

Figura 1. Diagrama de integración de módulos del sistema Consola Retro SDK



9. Solución de problemas comunes

9.1 El servicio no arranca

Revisar el log del servicio y la aplicación:

```

sudo systemctl status consola-retro.service
cat ~/consola_retro.log

```

9.2 El USB no es detectado

Verificar que pyudev está instalado con `python3 -c 'import pyudev; print(pyudev.__version__)'`. Si no está disponible, el sistema usa polling (hasta 2 segundos de retraso). Verificar que el usuario pertenece al grupo plugdev con `groups $USER`.

9.3 El USB se detecta pero no se monta

Verificar que `/etc/sudoers.d/consola_retro` existe con las reglas de mount y umount sin contraseña, y que el directorio `/media/pi/usb_retro` existe con permisos correctos.

9.4 El sistema no arranca automáticamente

Verificar que el servicio está habilitado con `systemctl is-enabled consola-retro.service`. Verificar que no existe `/boot/NO_CONSOLA` ni `/boot/firmware/NO_CONSOLA`.

10. Evidencias

Repositorio del proyecto: https://github.com/nasaruke/SDK_RetroGames

Video demostrativo: <https://youtu.be/pv0NvIFaCqE>

11. Conclusiones

En este proyecto se logró integrar una consola retro utilizando una Raspberry Pi como plataforma principal. A través del archivo `main.py` fue posible coordinar los módulos de pantalla, entrada, galería, emulador, arranque y monitoreo USB, manteniendo una estructura organizada donde cada componente cumple una función específica dentro del sistema. Como podemos describir para esta sección.

El uso de `systemd` permitió que la consola iniciara automáticamente al encender la Raspberry Pi, comportándose como una consola dedicada. La directiva `ConditionPathExists` para el modo de mantenimiento resultó ser un mecanismo elegante que no requiere modificar la configuración del servicio.

La detección de USB con `pyudev` resultó más eficiente que el polling: el hilo de monitoreo consume recursos de CPU prácticamente nulos mientras no hay eventos. El `threading.Lock` garantizó la ausencia de condiciones de carrera entre el hilo de monitoreo y el loop principal de la galería.

Finalmente, el reto de crear el script `instalar.sh` fue particularmente especial, ya que pocas veces hemos implementado este tipo de archivos, integrando prácticamente todo el proyecto aquí, lo que permitió que la configuración completa del sistema fuera de forma reproducible en cualquier Raspberry Pi con Raspberry Pi OS Lite, reduciendo errores manuales y garantizando que el entorno sea idéntico en cada instalación.

12. Cuestionario

La Tabla 3 contiene el cuestionario de evaluación del proyecto.

Tabla 3. Cuestionario de evaluación

#	Pregunta
1	¿Por qué se utiliza doble buffer (<code>display.flip</code>) en lugar de actualizar la pantalla píxel a píxel?
2	¿Qué ventaja tiene encapsular todas las llamadas a <code>pygame</code> dentro de la clase <code>Pantalla</code> en lugar de llamarlas directamente desde cada módulo?
3	¿Por qué se define un umbral de 0.5 para los ejes analógicos en lugar de detectar cualquier valor distinto de cero? ¿Qué problema resuelve esto en la práctica?
4	Describe el problema del framebuffer compartido entre <code>pygame</code> y <code>mednafen</code> . ¿Por qué es necesario llamar a <code>pygame.display.quit()</code> antes de lanzar <code>mednafen</code> y cómo se recupera la pantalla al regresar?
5	¿Qué función cumple el <code>threading.Lock</code> en la clase <code>MonitorUSB</code> ? ¿Qué condición de carrera podría ocurrir si se eliminara el lock?
6	¿Por qué se utiliza <code>pyudev</code> para detectar la inserción de USB en lugar de revisar periódicamente el contenido de <code>/dev</code> ? Describe las ventajas del enfoque basado en eventos sobre el polling.

13. Bibliografía

[1] E. A. Lee y S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2.a ed. Cambridge, MA: MIT Press, 2017.

- [2] Raspberry Pi Foundation, "Raspberry Pi OS Documentation," 2024. [En línea]. Disponible en: <https://www.raspberrypi.com/documentation/>
- [3] pygame Developers, "pygame Documentation," 2024. [En línea]. Disponible en: <https://www.pygame.org/docs/>
- [4] SDL Community, "SDL2 Wiki," 2024. [En línea]. Disponible en: <https://wiki.libsdl.org/SDL2/>
- [5] M. Kerrisk, The Linux Programming Interface. San Francisco, CA: No Starch Press, 2010.
- [6] Python Software Foundation, "threading — Thread-based parallelism," Python 3.11. Disponible en: <https://docs.python.org/3/library/threading.html>
- [7] Lennart Poettering y Kay Sievers, "systemd System and Service Manager," freedesktop.org, 2024. Disponible en: <https://www.freedesktop.org/wiki/Software/systemd/>
- [8] pyudev Contributors, "pyudev Documentation," 2024. Disponible en: <https://pyudev.readthedocs.io/>
- [9] Mednafen Team, "Mednafen Documentation," 2024. Disponible en: <https://mednafen.github.io/documentation/>